LUMYVUE / PrecisionSkin: complete technical record

AI-assisted facial skin screening research prototype: skin-disease classifier, Grad-CAM++ explanations, and YOLOv8 lesion detectors (acne, hyperpigmentation)

Compiled 22 October 2026 from eight working documents (results for the paper, methods and limitations notes, progress summary, project summary, training explained for the defense, panel study guide, ensemble review, and the run-the-app guide) plus the repository itself. Where the older documents disagreed with later work, the later, measured result is the one reported as current, and the older number is kept only in the history sections and labelled **superseded**.

How to read this document. Part 1 is the system and the final installed results (read this first). Parts 2 to 5 are the classifier, the explanation service and the detectors in detail. Part 6 explains why there are three models per condition. Part 7 is the code reference (every program and what it does). Parts 8 to 11 are the limits, ethics, defense questions and open items. The appendices hold the setup guide, the skin-tone model plan and the published-context table.

Status marks. Numbers were measured unless marked **NOT MEASURED**. Nothing in this document is a clinical claim: the system is a research prototype and a screening aid, not a diagnostic device and not a medical device.

Contents

1. The system and the final installed results
2. The skin-disease classifier (MobileNetV2)
3. Grad-CAM++ (explaining the classifier)
4. Detector data: sources, audit and label decisions
5. The lesion detectors: how they were built and what they scored (5A first detector, 5B acne, 5C hyperpigmentation, 5D eczema, 5E skin tone models)
6. Why three models per condition (ensemble review)
7. Code reference: the app, the scripts, the notebooks, the model files
8. Limitations
9. Ethics and data handling
10. Defense preparation (questions and answers, things not to say)
11. Mistakes, corrections and open items

- Appendix A: Run the whole app on a new laptop
- Appendix B: Monk Skin Tone (MST) model plan
- Appendix C: Published context for acne detection and paper wording

Part 1. The system and the final installed results

1.1 What the system is

LUMYVUE (PrecisionSkin) is a kiosk application: a WPF app (.NET Framework 4.7.2, C#) running on a CPU-only Windows mini PC (Acer, 11th-generation Intel Core i5, 16 GB RAM, NVIDIA GT 1030 2 GB which is not used for inference), with a Logitech BRIO camera. It takes three standardised views of a face (front, left, right) and does the following:

1. **Classifier (MobileNetV2).** Decides which of four classes the photo looks like: Acne, Hyperpigmentation, Eczema, Normal. Runs in the app through ONNX Runtime.
2. **Grad-CAM++ service.** A local Python/TensorFlow service (127.0.0.1:8765) that shows which image regions influenced the classifier. It is an explanation of the classifier, not a lesion map.
3. **YOLOv8 lesion detectors.** Draw candidate boxes where acne spots or hyperpigmentation patches are. The eczema detector is being trained by a teammate and is not installed.
4. **Study database.** Participants, consent, scans, researcher verification and dermatologist verification are stored in a local SQLite database. AI rows are append-only and are never modified by human review.

The two kinds of visual evidence are never merged into one claim: the heatmap is not a lesion detector and the detector is not an explanation of the classifier.

Approved objectives (current wording): CNN classification of acne, eczema and hyperpigmentation; standardised capture hardware; OpenCV preprocessing; evaluation with accuracy, precision, recall and F1; ISO/IEC 25010 and TAM. The mimicker logic was removed from the objectives and the Fitzpatrick AI module was removed from the system (see 5E).

1.2 What is installed in the app (`SkinDevApp/Model/`)

File	What it is
`precisionskin.onnx`	MobileNetV2 four-class classifier, 224 px, about 5.4 MB
`lesion_detector_acne.onnx` , `_m2.onnx` , `_m3.onnx`	three YOLOv8s acne detectors: seed 123, seed 42, seed 7 (about 44.8 MB each)
`lesion_detector_acne.json`	settings: imgsz 800, default confidence 0.275, NMS IoU 0.6, SHA-256, seeds, metrics, ensemble list
`lesion_detector_hyperpigmentation.onnx` , `_m2.onnx` , `_m3.onnx`	three YOLOv8s hyperpigmentation detectors (H25): seed 42, seed 7, seed 123
`lesion_detector_hyperpigmentation.json`	settings: imgsz 800, default confidence 0.225, NMS IoU 0.6, SHA-256, ensemble list

The app runs the three files of a condition on the same photo and fuses their boxes (weighted box fusion, IoU 0.5, confidence = mean of the three). Boxes outside the detected face region are dropped.

Housekeeping note on the acne `.json` : its `test_mean` block (mAP@0.5 0.413, F1 0.478) is the earlier single-seed-mean figure; the correct 3-seed average (0.439 / 0.507) is in its `note` line.

1.3 Final results at a glance

	Acne detector	Hyperpigmentation detector
Installed	3 seeds, YOLOv8s, 800 px, threshold 0.275	3 seeds (H25), YOLOv8s, 800 px, threshold 0.225
Training data	1,145 photos, 11,621 boxes, all boxed by hand	808 photos hand-corrected under rule v2 (each shown 3 times per epoch) + 1,260 de-duplicated public photos (3,684 training images)
Validation	154 photos	60 photos (rule v2)
Cleanest test	**fresh test, 100 photos** (36 with acne, 64 without): mAP@0.5 **0.336**, F1 0.431 (range 0.30 to 0.55)	**100 photos** boxed from scratch under rule v2, scored once: mAP@0.5 **0.508**, mAP@0.5:0.95 0.209, precision 0.590, recall 0.465, F1 **0.520** (range 0.44 to 0.59)
Other test	development test 167 photos (scored 3 times): mAP@0.5 0.439, F1 0.507 (range 0.470 to 0.544), zone F1 0.832	3x3 face-zone F1 0.608; a patch found in 93 % of photos
False alarms on photos without the condition	39 % of the 64 no-acne photos get a box (15 % on the 27 clear-skin photos, 57 % on the 37 acne-pool negatives)	about 32 % on clear skin
Single seeds on the same test	fresh test 0.292 / 0.285 / 0.320; development test 0.419 / 0.412 / 0.408	0.440 / 0.450 / 0.410
Label ceiling (same annotator, IoU 0.5)	F1 0.30	F1 0.49 to 0.55 under rule v2 (0.28 under the first rule)

Classifier (1,604 test photos, never used for choosing): accuracy 0.926 +/- 0.005 over 5 seeds, macro-F1 0.914 +/- 0.005; the selected seed scored accuracy 0.929 and macro-F1 0.917. Its weakness: about 13.5 % of acne photos are predicted as eczema.

1.4 The story in one page (what we did, in order)

1. **Classifier.** Trained a MobileNetV2 on 12,000 balanced facial images with 5 seeds, chose the seed on validation, tested once, exported to ONNX and verified the export against Keras.
2. **Explanations.** Built the Grad-CAM++ service, proved the four class maps are computed correctly but are 87 to 99 % alike (a property of the classifier), and showed they are attribution, not localisation.
3. **First detector.** Audited 17,017 public Roboflow images (5,731 unique photo groups), trained a three-class YOLOv8n (mAP@0.5 0.35). Tried larger input and an eczema extension: no improvement.
4. **Own labels.** Because the public boxes were rough, we built a local box tool and the researcher boxed acne and hyperpigmentation by hand with one rule per condition, with model-drafted boxes only as a helper.
5. **Specialist detectors.** Trained one-class YOLOv8s detectors for acne and hyperpigmentation, installed them in the app as specialists.
6. **Diagnosis.** A read-only diagnosis of data and models found what limits the scores: incomplete acne labels, tiny spots, public hyperpigmentation boxes twice as coarse as ours, and annotator inconsistency.

7. **Hyperpigmentation rescue.** Measured label reproducibility (F1 0.28), wrote annotation rule v2, relabelled validation and test from scratch, converted and hand-corrected training labels, added public photos, trained 3 seeds: mAP@0.5 from 0.18 to 0.508.
8. **Acne experiments.** Ran Phase 1 to 3 with a decision rule fixed beforehand, built a fresh untouched test set, and found that the 960 px candidate's gain did not hold. The installed 3-seed 800 px model stayed.
9. **Honest documentation.** Built a read-only evidence notebook for both detectors, tested merging seeds into one ONNX (lower quality, not used), reviewed whether the three-model design is justified, cleaned the repository and committed it.

Part 2. The skin-disease classifier (MobileNetV2)

2.1 Facts

Item	Value
Task	single-image, 4-class classification: Acne, Hyperpigmentation, Eczema, Normal
Data	12,000 images, **3,000 per class**; split **8,793 train / 1,603 validation / 1,604 test** (split by duplicate group, so look-alike photos stay in the same split). Manifest `master_dataset.csv`
Facial quality	every image passes the project's facial QA (`facial_primary` 10,367, `facial_aux_train` 1,633). "Facial" includes tight lesion-region crops: full face 1,346, partial face 3,324, close-up facial 2,221, lesion-region facial 4,266, unknown 830, non-facial 13
Run used	`facial_only` (non-facial-origin eczema photos, 1,633 images, kept out of training; eczema trains on 913 facial photos); natural class counts with **class weights** (eczema 1.96, others about 0.86); no oversampling
Model	**MobileNetV2**, ImageNet-pretrained, 224 x 224 input, then global average pooling, BatchNorm, Dropout 0.4, Dense 256 (L2 1e-4), BatchNorm, Dropout 0.4, Dense 128, BatchNorm, Dropout 0.4, Dense 4 softmax (float32)
Training	**Phase A:** backbone frozen, 30 epochs, learning rate 1e-3. **Phase B:** fine-tune from layer 100 on, 15 epochs, learning rate 1e-5. Batch 16. Early stopping and learning-rate reduction on validation loss. **5 seeds (0 to 4)**
Augmentation	conservative, training only: horizontal flip, brightness/contrast, small noise, coarse dropout, resize
Model choice	the seed with the best **validation** macro-F1 = **seed 1 (0.9272)**; the test set was touched once
Export	Keras to **ONNX** (5.4 MB, 224 x 224, RGB, normalization `(pixel / 127.5) - 1`); Keras and ONNX agree on 48 test images (largest probability difference 0.0044)
Speed (GPU, measured)	ONNX inference about 20 ms; Grad-CAM++ about 300 ms. **CPU mini PC not measured for the classifier**
Where the code is	notebook `Pipeline V5 - With Facial Eczema.ipynb` (executed, 47 cells, run in order); run folder `runs/facial_only/` (checkpoint audit, calibration report, ONNX report); app side `Aiengine.cs` and `Onnxinference.cs`

The app's preprocessing must stay byte-identical to the Grad-CAM service: BGR to RGB, resize to 224 x 224 (bilinear), float32, then `(p / 127.5) - 1.0`. The class order is fixed by the trained model (Acne 0, Hyperpigmentation 1, Eczema 2, Normal 3) and must never be reordered.

2.2 Results (test set, 1,604 photos, never used for choosing)

- **5 seeds:** accuracy **0.926 +/- 0.005**, balanced accuracy 0.924, macro-F1 **0.914 +/- 0.005**, kappa 0.900.
- **Selected seed (1):** accuracy 0.929, macro-F1 0.917. 95 % ranges: accuracy 0.916 to 0.941, macro-F1 0.902 to 0.931.
- **Per class (precision / recall / F1):** Acne 0.96 / 0.85 / 0.90; Hyperpigmentation 0.97 / 0.95 / 0.96; Eczema **0.74** / 0.93 / 0.82; Normal 0.98 / 0.98 / 0.98.
- **Calibration:** validation expected calibration error 0.012 (the confidence numbers are meaningful).

- **Leak-free check:** 488 test photos had a same-source sibling in training. On the other 1,116 photos: accuracy **0.909**, macro-F1 **0.912**. The headline is slightly optimistic; quote both.

2.3 The main weakness (say it before being asked)

Acne is often called eczema: 59 to 63 of 465 acne photos (about 13.5 %) were predicted as eczema, which is why eczema precision is only 0.74 and acne recall is 0.85. Many of these errors are confident (median confidence 0.77). The kiosk shows a result only above a confidence gate of **0.65**, which already filters about 25 % of those acne-as-eczema errors. A simulated "ambiguous" guard would catch only about 14 % of them and was not applied. Class weights raise eczema recall (0.93) at the cost of precision.

2.4 Scan flow in the app

The scan is a state machine (`ScanStateMachine.cs`): Live, Scanning, Stabilizing, HoldStill, StableCapturing. A prediction stabiliser (`Predictionstabilizer.cs`) smooths the live output, `FrameQuality.cs` gives the guidance state (face box fraction, lighting), `ViewPoseEstimator.cs` checks that the patient is really showing the requested view (YuNet face detector landmarks), and `ScanPipeline.cs` runs the final analysis of one captured frame (classifier, Grad-CAM++, detectors). `MultiViewSession.cs` ties Front, Left and Right captures to one session, and `CaptureArchive.cs` writes one folder per capture. Captures are never copied into a training set automatically.

Part 3. Grad-CAM++ (explaining the classifier)

3.1 Service and what was verified

Service `gradcam_service.py` (v2.0) loads `precisionskin_best.keras` and answers on 127.0.0.1:8765. A startup parity check requires the Grad-CAM forward path to reproduce `model(x)` (it fails fast otherwise; the tolerance was relaxed from 1e-4 to 5e-3 because it refused to start on other CPUs).

Question raised: the four class maps looked almost identical. What was verified:

- Each class map uses the gradient of **its own** pre-softmax logit (Acne 0, Hyperpigmentation 1, Eczema 2, Normal 3). The all-class path was compared with four separate single-class computations: **maximum difference = 0**.
- C# side (`Gradcamservice.cs` , `ClassHeatmapRenderer.cs`): each map is decoded by class index and coloured with its own colour; colour does not decide where a map appears.

3.2 Why the maps still look alike (measured on 12 captures)

Raw (un-normalised) class CAMs, mean pairwise Pearson correlation:

Target layer	Grid	Mean correlation between classes
`Conv_1` (used)	7x7	0.947
`out_relu`	7x7	0.954
`block_13_expand_relu`	14x14	0.984
`block_6_expand_relu`	28x28	0.989
`block_3_expand_relu`	56x56	0.992

Earlier layers give finer maps but make the classes **more** alike (they respond to image structure, not class evidence). The head is global-average-pool plus three Dense blocks, so every class map is a re-weighting of the same activation maps (cosine similarity of the Acne, Hyperpigmentation and Eczema output weights: 0.29 to 0.53). A "this class minus the others" map did not help on real captures (a class with a 1 % score got the strongest map), so it is opt-in only.

Conclusion to state: the maps are correctly computed class-specific attributions. Their similarity is a property of the trained classifier (it relies on shared facial features for the three skin conditions), **not** a bug.

3.3 Grad-CAM++ is not localisation (enrichment test)

On the 167 hand-boxed acne photos, the acne Grad-CAM map put **1.30x** the chance of its heat inside the researcher's boxes, versus **1.18x** for a fixed face-centre blob. It carries almost no lesion-specific information. A highlighted region is not proof of affected skin.

3.4 App display changes made from this

One Grad-CAM++ picture (the detected class) in the record, gallery and print views (all four are still saved); overlay opacity follows the class score; each map is normalised separately (compare shape, not brightness); shorter on-screen notes. 38 of 42 existing scans were re-rendered to the new look (backups in

`_before_display_update`; originals, heatmaps and records untouched; 4 old scans without heatmap files were skipped). Tool: `gradcam/` scripts and `gradcam_enrichment.py`.

Part 4. Detector data: sources, audit and label decisions

4.1 Public sources (all Roboflow Universe exports, YOLO format, licence CC BY 4.0 as stated in each `data.yaml`)

Source folder	Images	Unique photos*	Boxes	Original labels
ACNE Detection v2i (BEAUTYAI, `acne-detection-mph4m` v2)	6,257	~1,800	80,950	`0`
acne-detection v2i (Sarah Kiyani)	4,859	~2,350	31,409	`0,1,2,acne,pimple,spot`
hyperpigmentation yolov10 v3	2,250	~770	5,271	`hyperpigmentation`, `redness` (1 box)
Pigmentation v1i (Tinnaluris Workspace)	1,634	~960	5,293 polygons	`0`
hyperpigmentation v2i (Goat)	1,169	~300	2,850	`Hyperpigmentation`
Hyperpigmentation v1	500	~380	2,014	`Hyperpigmentation`, `3`
Eczema v1 facial	348	~332	714	`Eczema`
Total audited	**17,017**	**5,731 duplicate groups**	123,208 boxes + 5,293 polygons	

*per-source unique counts come from name grouping; the authoritative number is the duplicate-group count after cross-source merging. All boxes were drawn by the original dataset annotators; polygons (Pigmentation v1i) were converted to enclosing boxes. Exact project names, URLs and citations still have to be added to the reference list.

4.2 Label decisions (each checked by eye on sample images)

Source and original label	Decision	Evidence
ACNE Detection `0`	Acne	pimples and blemishes (many grayscale or noisy augmented copies)
acne-detection v2i `0`, `1`, `2`	Acne	small pimple-like spots on faces
acne-detection v2i `spot`	Acne (**least certain**)	small blemish-like boxes; a few may be pigmentation
Hyperpigmentation v1 `3`	Hyperpigmentation	genuine melasma and sun-spot patches
Pigmentation v1i `0`	Hyperpigmentation	brown cheek patches
yolov10 `redness`	dropped	1 box in the whole dataset

The original label, source and mapping history are kept for every annotation (`annotations_long.csv`).

4.3 The dataset audit (notebook `PrecisionSkin_Localization_Dataset_Audit`, originals read-only)

- 0 corrupted images; 34 images with no annotation; 0 annotation files without an image.
- **Duplicates.** Augmented copies (flips, rotations, grayscale, noise) and the same photo in several datasets are common (95 % of images are in a multi-image group). A first method (64-bit perceptual hash, Hamming <= 6) both missed real copies and chained unrelated photos into one 6,364-image group. It was replaced after calibration on the data: **32x32 grayscale thumbnail correlation >= 0.97 against all 8 rotations and flips**, plus identical-file and same-Roboflow-name links, with a safety cap of 150 images per group. Result: **5,731 groups, largest 51 images, 642 groups spanning several sources, 23 groups (295 images) containing the same photo under two conditions** (excluded from training and evaluation).
- **Anatomical site.** A face detector (YuNet) found a face in 68 % of images (11,645 of 17,017); the rest are mostly cheek or nose close-ups. After browsing about 16 random images of each source, **six sources were declared facial by the researchers**. Roughly 1 in 6 sampled images of the two acne sources and Pigmentation v1i looked unclear (could be neck or back). This is a documented assumption.
- **Flagged for review:** 504 images (295 same photo under two conditions, 115 dark-vignette or dermoscopy-like, 34 very large box, 34 empty label file, 22 low resolution, 8 zero-area box). **16,513 kept.**

4.4 Later rule: more rigorous near-duplicate checks

For the specialist detectors the duplicate check was tightened to thumbnail correlation >= 0.90 against 8 rotations and flips, applied between every training, validation, test, independent and fresh set.

Part 5. The lesion detectors: how they were built and what they scored

5.1 Shared training protocol

- **Model:** one-class YOLOv8s (official Ultralytics: 8.4.171 in the assistant's `.venv_det`, py3.11; 8.4.172 in the researcher's VS Code py3.13 environment), COCO-pretrained `yolov8s.pt`, 800 px, batch 8, AdamW, up to 100 epochs, cosine learning-rate decay, mosaic off for the last 10 epochs, flips, small rotations, scale and colour jitter, early stopping after 30 epochs without improvement (Ultralytics stops on fitness = 0.1 mAP50 + 0.9 mAP50-95). Seeds 42, 7 and 123.
- **Hardware:** NVIDIA RTX 4060 Laptop GPU (8 GB), WSL2 with about 6.7 GB RAM; never two trainings at once; resumable through `last.pt`.
- **Selection discipline:** every choice and every threshold uses **validation photos only**. Test sets are scored once, with a lock file. A change is adopted only if it passes a rule fixed beforehand: +0.02 mAP@0.5, precision and recall not lower by more than 0.02 (plus false alarms not more than 5 points higher on the fresh test).
- **Metrics:** mAP@0.5 and mAP@0.5:0.95 (our scorer, cross-checked with Ultralytics' own validator: 0.421 vs 0.413 on the same photos), precision, recall and F1 at the validation-chosen confidence, 95 % ranges by resampling test photos (1,000 times), 3x3 face-zone F1, photo-level and count-level results, and false alarms on clear skin.
- **Training goes through notebooks** that are executed so the epoch output stays saved (`..._WITH_OUTPUTS.ipynb`).
- **Deployment:** export to ONNX (opset 12, fixed 800 px, batch 1), write the `.json` with threshold, SHA-256 and metrics (`install_specialist_detector.py`), run in the C# app through ONNX Runtime, fuse three files by weighted box fusion.

5.2 5A. The first detector (public data, three classes) and early experiments

Dataset construction (notebook `PrecisionSkin_Detector_Export_and_Train`). Removed zero-area boxes, corrupted and low-resolution images, photos under conflicting conditions (324 images) and boxes thinner than 3 px (52). Split **by duplicate group** (asserted in code), seed 42. Validation and test contain facial, kept images only, one image per photo (highest-resolution copy). Training: at most 3 copies per photo; eczema training images used 3x. Test set: **566 photos: acne 370, hyperpigmentation 164, eczema 32.**

Run A (deployed at the time). YOLOv8n, COCO-pretrained, 640 px, batch 16, up to 100 epochs, patience 25, cosine LR, mosaic closed for the last 10 epochs, flip 0.5, rotation +/-10, scale 0.5, HSV jitter (0.015/0.5/0.35), seed 42. Stopped at epoch 82 (best epoch 66), about 1 h 25 min.

Class (test)	Photos	Precision	Recall	mAP@0.5	mAP@0.5:0.95
Acne	370	0.50	0.35	0.36	0.11
Hyperpigmentation	164	0.42	0.37	0.32	0.13
Eczema	32	0.35	0.46	0.37	0.13
All	566	0.42	0.39	0.35	0.12

Best F1 0.39 at confidence 0.175. CPU latency (ONNX, 4 threads, laptop): median 31.5 ms. Boxes land on real spots, patches and an eczema patch, but many lesions are missed.

Runs B and C (larger input, longer training): no improvement. B (960 px, 79 epochs): validation mAP@0.5 0.319. C (640 px, full 100 epochs): 0.333 at best epoch 66, 0.299 at the last (overfitting after epoch 66). Run A stayed.

Eczema extension v2: negative result, v1 kept. The v1 detector drafted eczema boxes (confidence >= 0.20) on 338 more unique facial eczema photos; the researchers corrected every image. Draft quality: 96.7 % of draft boxes kept (precision) but only 36.1 % of the final boxes found (recall); 326 images (895 boxes) entered a new training-only source. Validation and test were pinned to the same 566 + 566 photos as v1.

Class	Metric	v1 (deployed)	v2	Change
All	mAP@0.5	0.349	0.303	-0.046
All	mAP@0.5:0.95	0.122	0.107	-0.015
Acne	mAP@0.5	0.357	0.358	0.000
Hyperpigmentation	mAP@0.5	0.323	0.312	-0.011
Eczema (n = 32)	mAP@0.5	0.368	0.241	**-0.127**

Interpretation (hypotheses, not tested): drafts came from v1, so lesions it missed were probably left unboxed; training on partly unboxed images teaches the model that real lesions are background. With 32 eczema test photos and one seed, part of the change may be noise.

The decision that changed everything. Compared with published work and reviewed by eye, the public boxes were rough, and the first detector learned the public box style. The researcher decided to box acne and hyperpigmentation by hand with one written rule per condition (Phase B below), using a local box tool.

5.3 5B. Acne

Timeline

1. **Tools.** `lesion_box_tool.py` (local web editor, X = "no lesion", saves YOLO labels), the acne sprint (all 4,073 distinct acne photos from the researcher's folder, de-duplicated at 0.93, test photos reserved), and the **pre-draw helper** (`acne_predraft.py`): a model trained on the researcher's own boxes draws first boxes that are then corrected. Measured reliability of drafts: 87 % recall and 98 % precision (later More_Faces: 79 % and 99.5 %). Drafting was faster than boxing from scratch (14.2 s per photo).
2. **Dataset v1 to v4** (`merge_acne_dataset.py`): only photos the researcher reviewed are used; leak checks; validation boxed from scratch only. Reviewed 400 helper-drafted non-facial photos (v3): **the test score dropped** to 0.387; the facial-only ablation v3a recovered it (0.423); those drafts were removed. The researcher then boxed **More_Faces** (580 pre-filtered photos: 448 boxed, 130 marked X, 4,662 boxes) giving **dataset v4: 1,145 train / 154 validation / 167 test photos**.
3. **Final v4 run:** 3 seeds (42, 7, 123), YOLOv8s, 800 px (notebook `Pipeline V5 - Lesion Detector`, executed by the researcher on 2026-10-10 and kept in `acne/evidence/` unchanged, SHA-256 b005e0c9...fe59c).

History on the same 167 hand-boxed test photos (superseded stages)

Model	train photos	mAP@0.5	mAP@0.5:0.95	P	R	F1
v1 (3-class, in the app before)		0.220	0.066	0.36	0.23	0.28
first new model (one seed)	708	0.448 (lucky seed)	0.135	0.45	0.56	0.50
v3 (+400 drafted non-facial)	1,099	0.387	0.116	0.43	0.53	0.47
v3a (facial only)	748	0.423	0.127	0.42	0.55	0.48
v4, 3 seeds, mean of individual seeds	1,145	**0.413** (std 0.006; range 0.37 to 0.46)	0.122	0.43	0.54	0.48

Gain over v1: +0.193 mAP@0.5 (95 % range +0.147 to +0.238). The honest single-run figure is 0.413, not the lucky 0.448.

Reliability at the level the app is used (3 seeds, then-threshold 0.35)

Photo level (167 acne photos vs 200 healthy photos from the Normal class): AUC **0.971**, sensitivity 95.0 %, specificity 85.0 % (healthy photos not individually verified). Count level: Spearman 0.70, mean error 3.9 boxes per photo (about 8 true). 3x3 zones: precision 0.83, recall 0.79, F1 0.81. These do not replace mAP: box-level mAP stays about 0.4 because acne spots are tiny and hand-boxed edges differ by a few pixels.

The acne detector is not acne-specific

At the app threshold it puts boxes on 54 % of eczema-class photos (n = 341, many non-facial or papular), 29 % of hyperpigmentation-class photos and 15 % of healthy photos (labels not verified). Describe it as a candidate-lesion (spot or papule) marker, not an acne diagnosis tool.

Independent set (scored once, frozen protocol)

41 blind-boxed photos (437 boxes), best-validation seed, threshold 0.26: mAP@0.5 **0.358** (range 0.31 to 0.44), precision 0.38, recall 0.52, F1 0.44. Lower than the development test (0.413) by 0.055: report it as the more conservative figure.

Diagnosis: why acne scores are where they are

Data are clean (no near-duplicates, no doubled boxes, consistent sizes). The limits are: tiny spots (recall about 20 % for boxes under 2 % of the photo side and 40 % at 2 to 4 %); **incomplete labels** (in 60 photos re-reviewed with model help, boxes rose from 893 to 1,430, +60 %); strict-IoU sensitivity on about 30 px boxes (mAP@0.5:0.95 about 0.12 whatever the model); mild overfitting after epoch about 50. Same-annotator reliability on a blind re-box of 30 validation photos: **F1 0.30 at IoU 0.5**, so strict box-level scores are capped. Box-scale and margin checks (`acne_box_scale_check.py` , `acne_box_margin_check.py`) showed the disagreement is mostly about where edges go on tiny spots, not one consistent size convention.

Controlled experiments (validation only, seed 42; baseline mAP@0.5 0.425, F1 0.494)

Exp	Change	mAP@0.5	Verdict
A1	+ 115 healthy-skin negatives	0.431	noise
A2	milder colour jitter	0.424	noise
A3	box-loss gain 10	0.431	noise

Exp	Change	mAP@0.5	Verdict
A4	lower learning rate	0.433	noise
A5	960 px input	0.442	+0.017, just under the +0.02 rule
A6	YOLOv8m	(acne 0.426 vs 0.425 baseline)	no gain
A7	60 relabelled photos (missed spots added)	validation labels incomplete, so precision may even fall	not adopted alone

More photos (708 to 1,145) did not help either. The seed spread of the baseline on validation is about 0.01.

Phase 1 to 3 (the notebooks the researcher ran)

- **Phase 1** (`PrecisionSkin_Acne_Phase1_Experiments.ipynb`): E1 to E4, validation only, seed 42, same rule. No data or label change passed the pre-set rule.
- **Phase 2** (`..._Phase2_FinalTraining.ipynb`): the best configuration (E1, 960 px, lower learning rate) times 3 seeds, 3-model average, threshold on validation, development test once. 960 px candidate: validation mAP@0.5 0.471 vs 0.460, development test 0.481 vs 0.439.
- **Phase 3** (`..._Phase3_FreshTest.ipynb`): the **fresh untouched test set** (below), both models scored once each at their own validation threshold, with the decision rule.

Three-seed average installed (2026-10-13)

Threshold 0.275 = best F1 on the 154 validation photos, fixed before scoring. Development test (167 photos): mAP@0.5 **0.439**, precision 0.465, recall 0.556, F1 **0.507** (range 0.470 to 0.544), zone F1 0.832, a box in 98 % of photos. The single installed seed (123): 0.419, F1 0.482; seeds 42 and 7: 0.412 and 0.408. Gain +0.02 mAP@0.5 and +0.025 F1; the F1 ranges overlap, so the gain is small. The development set has now been scored several times, so its figures are mildly optimistic. The app's own C# detector code reproduces the offline score (F1 0.506, zone F1 0.829).

Fresh untouched test set and the 960 px candidate (2026-10-15)

- **Set:** 100 photos boxed from scratch (no model drafts), never used for training, validation, tuning or thresholds, checked against every other photo for near-duplicates: **36 photos with acne (203 boxes) and 64 without** (marked X; 37 from the acne pool, 27 from the clear-skin pool). A first notebook run scored only the 36 positive photos because of a folder-selection bug; it was archived and the corrected run on all 100 photos is the reported result. Nothing was tuned between the runs.
- **Result (all 100 photos):** installed mAP@0.5 **0.336**, mAP@0.5:0.95 0.088, precision 0.350, recall 0.562, F1 0.431 (range 0.30 to 0.55), zone F1 0.695. Candidate 960 px: mAP@0.5 0.300, mAP@0.5:0.95 0.077, precision 0.301, recall 0.571, F1 0.395 (0.28 to 0.50), zone F1 0.667. Single seeds: installed 0.292 / 0.285 / 0.320; candidate 0.279 / 0.256 / 0.268. Positive photos only (36): installed 0.418, F1 0.498; candidate 0.389, F1 0.471.
- **False alarms on the 64 no-acne photos:** installed 39 %, candidate 50 %. By source: installed 57 % on the 37 acne-pool negatives and 15 % on the 27 clear-skin photos; candidate 59 % and 37 %.
- **Pre-set decision rule:** failed (mAP -0.036, precision -0.048, false alarms +10.9 points). **The installed model stays.** The development-test gain of the candidate did not hold on fresh photos. The goal of 0.50 mAP@0.5 was not reached (best 0.336 on all 100 photos); the fresh set is small (203 boxes), so differences of about 0.03 are within noise.

Other acne checks

Display-gate rule (`acne_gate_rule.py` , show boxes only if at least N boxes above threshold), box count versus true count, Ultralytics-validator cross-check (`acne_verdict_checks.py`), CPU latency and memory, and a training run with healthy-skin background images (`train_acne_v5.py` ; Ultralytics recommends 0 to 10 % background images) which did not help beyond noise.

5.4 5C. Hyperpigmentation

The starting point (superseded)

Phase A built `hyperpig_autolabel.py` (one-class dataset, duplicate checks, pseudo-labelling with resume, the scoring code used by later notebooks) and the AutoBox and Boxing Sprint notebooks. The researcher hand-boxed hyperpigmentation: **646 photos opened = 504 boxed (2,117 boxes) + 142 marked X**; 100 hand-boxed photos were held out as the test set.

Stage (old box style, 100 hand-boxed test photos)	mAP@0.5	mAP@0.5:0.95	P	R	F1
v1 (3-class, in the app before)	0.053	0.019	0.19	0.05	0.08
s_800 (installed at the time)	0.178	0.051	0.29	0.21	0.24
final old-style, 3-seed mean	0.229 (std 0.020)	0.065	0.31	0.30	0.30
3-seed box average (old style)	0.278	n/a	0.37	0.34	0.354 (range 0.30 to 0.41)

(The n_640 and m_800 comparison runs stopped after 7 and 9 epochs and were invalid.) Scores in this old style are **not comparable** with rule-v2 scores because the target changed.

Diagnosis (what limited it)

Public boxes were about 2x coarser than the researcher's (they cover 18 % vs 9 % of the photo) and made up 75 % of training; the earlier validation set was in the public style; 2,524 of 3,031 training photos had near-duplicates; look-alikes between splits; 41 % of training photos and 51 % of the hand-boxed test photos are under 400 px; the researcher's own boxing style drifted by about 50 % within one session. Where it failed on validation: patches found at IoU 0.5: small 24 %, medium 37 %, large 51 %; matching looser raised F1 from 0.37 (IoU 0.5) to 0.47 (IoU 0.3) and 0.51 (IoU 0.2): many missed patches were found but not tightly, which is edge ambiguity of diffuse patches. Same-annotator reliability (30 validation photos boxed again blind): **F1 0.28 at IoU 0.5**, with 3.0 vs 6.4 boxes per photo. The detector agreed with the original boxes better (0.41) than the annotator agreed with themselves.

Annotation rule v2 (`HYPERPIGMENTATION_ANNOTATION_RULE_V2`)

One box per connected patch; patches that touch or are closer than the smaller patch's width count as one box; skip marks under about 7 % of the photo; one box stays inside one facial region; the edge is where the darkening fades to half; usually 1 to 5 boxes; no zooming in to find more. Two passes minutes apart agreed at F1 0.67 (an upper bound because of memory); on a day gap agreement was **0.49 to 0.55**. Rule-v2 boxes are about twice as large (median side 0.22 vs 0.125).

Relabelling under rule v2

- **Test:** 100 photos boxed from scratch by the researcher (226 boxes), blank, shuffled and renamed. **Validation:** 60 photos under rule v2 (30 + 30). Never used for training.
- **Training labels:** old boxes converted to rule v2 by code (`hyp_v2_transform.py` : merge boxes closer than 1.0x the smaller side, drop boxes under 0.09), giving H20 (1,115 images); then 808 photos hand-corrected (`make_hyp_v2_relabel.py` , `make_more_boxing.py` ; consistency checked by `hyp_v2_train_consistency.py`).
- **H20 result (converted labels, test = 100 rule-v2 photos):** 3-model average mAP@0.5 **0.341**, F1 0.409. The same recipe on old-style boxes scored 0.077 on these photos, and the previously installed model 0.207, so the target change was real.

The final model H25 (installed 2026-10-13)

Training: 808 hand-corrected rule-v2 photos (each shown 3 times per epoch) + 1,260 de-duplicated public photos (3,684 training images); YOLOv8s, 800 px, AdamW lr 0.0005, up to 100 epochs, seeds 42, 7, 123. Threshold 0.225 = best F1 on validation.

- **Validation (60):** single seeds 0.449 / 0.480 / 0.436 (mean 0.455, std 0.023); 3-model average mAP@0.5 **0.532**, mAP@0.5:0.95 0.194, F1 0.563.
- **Test (100, scored once, stored by the notebook):** mAP@0.5 **0.508**, mAP@0.5:0.95 **0.209**, precision 0.590, recall 0.465, F1 **0.520** (range 0.44 to 0.59); zone F1 0.608; a patch found in 93 % of photos; patch count mean error 0.94 patches, within 1 in 82 % of photos, Spearman 0.54. F1 by overlap needed: IoU 0.2: 0.67, 0.3: 0.65, 0.5: 0.52, 0.7: 0.28, 0.8: 0.13. Single seeds: 0.440 / 0.450 / 0.410.
- **Gain over the previous installed model** (same test photos): mAP@0.5 0.341 to 0.508, F1 0.409 to 0.520. The app's own C# code reproduces it (precision 0.608, recall 0.460, F1 0.524, zone F1 0.614).
- **False alarms on clear skin:** about 32 % (`negatives_eval.py`).

What helped, in order (validation, single seed unless stated)

1. Hand-corrected rule-v2 labels (H23/H24 vs converted H20): +0.09 mAP@0.5. The code conversion alone helped a lot; the first about 200 hand-corrected photos gave most of the gain; correcting beyond 200 added nothing measurable (H24 = H23).
2. Adding the de-duplicated public photos (H25 vs H24): +0.07.
3. Averaging three seeds: +0.05 on test.

What did not help

960 px (0.308), YOLOv8m (0.305), starting from the acne weights (0.278) or from the public-photo model (0.312), a second round of 230 reviewed drafts (0.296 vs 0.298), a lower learning rate (looked like +0.028 on one seed; the 3-seed mean 0.300 equals earlier runs), a five-model average (0.374 vs 0.357 on validation, within noise for five times the cost), and test-time augmentation tricks (`hyp_ensemble_tta.py`).

Limits (hyperpigmentation)

One annotator; rule v2 boxes are whole patches about twice the first rule's (scores not comparable with old results); the rule was re-checked only minutes apart (a one-day pass C is prepared, not done); no independent set under rule v2; boxes are candidate patches, not a diagnosis.

5.5 5D. Eczema (teammate's detector)

A teammate trains it. Handover materials: the lesion pipeline notebook with `CONDITION = "eczema"`, `install_specialist_detector.py --cls eczema`, the step-by-step guide "How we improved hyperpigmentation" and a detailed prompt for her AI assistant (both in `eczema/docs/`). Facts: v1's eczema data are 277 train photos, 32 validation, 32 test; 64 % of photos under 400 px. The recommended order: measure label reproducibility (a 30-photo blind re-box), decide whether photos are facial, compare box style per source, write a boxing rule, relabel evaluation sets from scratch, run one-change experiments on validation only, confirm with 3 seeds, score a fresh test once, measure false alarms on clear skin.

5.6 5E. Skin-tone models

Fitzpatrick AI model: discontinued (negative finding). A MobileNetV2 trained on a public face-photo set with six folders (Types I to VI, 3,000 images each): validation exact accuracy 51.9 %, within one type 79.8 %, Type I recall 0.009, Type IV recall 0.173; the test split was never scored. Types I and II could not be told apart (AUC 0.48; 78 % of both predicted as Type II) while Types II/III and III/IV separated almost perfectly (AUC 0.97 and 0.96) although median skin colour (ITA) does not support that order. The six folders follow apparent ethnicity groups, not measured skin tone, so the model learned appearance, not phototype. No train-validation leakage. The module was removed; it was never connected to the disease classifier or the detector, so those results are unaffected. Skin phototype in the app is only a researcher- or dermatologist-entered value.

Consequence for claims: the classifier and detectors were **not** evaluated per skin tone. Do not state they were validated across Fitzpatrick or Monk types. The Monk Skin Tone plan is in Appendix B.

Part 6. Why three models per condition (ensemble review)

6.1 What the three models are

They are **not three different architectures.** They are three copies of the same YOLOv8s (800 px, same COCO-pretrained start, same dataset, same settings) trained with three random seeds (42, 7, 123). The seed changes the random start of the new layers, the order of the photos and the random augmentations, so each copy ends with slightly different mistakes. The app runs all three and fuses their boxes (weighted box fusion, IoU 0.5, confidence = mean). A box that two or three copies agree on stays strong; a box only one copy believes in gets a lower fused score and is more likely to be dropped by the threshold.

This is **ensemble learning** (a seed ensemble or deep ensemble), a standard, published technique that reduces variance. It adds no new knowledge: the copies saw the same data, so diversity and gain are modest by nature.

Terms. *Single final model* = one weight file. *Final ensemble model* = a fixed set of weight files plus a fixed fusion rule and threshold, treated as one detector. *Multi-condition system* = acne detector + hyperpigmentation detector (+ eczema later) + classifier. Having six files does not mean there is no final model: the final model per condition is "3 seeds + fusion at IoU 0.5 + threshold X", frozen before the test was scored, and recorded in the `.json`.

6.2 Does the evidence justify keeping the ensembles?

mAP@0.5. "Best single" = the best of the three seeds on the same set.

Detector, set	Seeds alone	Ensemble	Gain over best seed	Notes
Hyperpigmentation, validation (60)	0.449 / 0.480 / 0.436	0.532	+0.052	threshold chosen here: optimistic
Hyperpigmentation, test (100, scored once)	0.440 / 0.450 / 0.410	**0.508**	**+0.058**	cleanest evidence
Hyperpigmentation, earlier old-style model	single mean 0.229	0.278	+0.03 to +0.05	same direction, different experiment
Acne, development test (167)	0.419 / 0.412 / 0.408	0.439	+0.020	set scored several times; F1 ranges overlap
Acne, fresh test (100)	0.292 / 0.285 / 0.320	**0.336**	+0.016	untouched set; ensemble beats every seed, gain small

Honest reading:

- The direction is consistent (5 of 5): the ensemble was never worse than its best member.
- The size differs. Hyperpigmentation: +0.05 to +0.06, clearly more than the seed spread (std about 0.02). Acne: +0.016 to +0.02, about the size of the seed spread, so for acne the gain is *suggestive, not proven*.
- **Missing statistic:** no paired bootstrap of the *difference* (ensemble minus best seed) has been run. The confidence ranges above are for each score alone and overlap, so "statistically significant" cannot be claimed. It can be computed from existing predictions without retraining.

- Thresholds were chosen on validation; the acne development test was scored several times; the fresh-test first run had a folder bug (documented, archived).
- Annotation limits (F1 0.30 for acne, about 0.5 for hyperpigmentation) mean small differences between models are partly unmeasurable.
- Not evaluated: false alarms per single seed, results by skin tone. Leakage: de-duplicated by thumbnail correlation across all splits; no evidence of leakage was found, and it was not re-run in this review.
- Diminishing returns: a 5-model average reached 0.374 vs 0.357 on validation (within noise), so 3 was chosen.

Verdict: justified for hyperpigmentation, weakly supported for acne.

6.3 Options compared

Nothing forces three files. The app can run one file with no code change (`LesionDetector.Peers` empty).

Option	Evidence we have	Pros	Cons / needs
1. Best single seed, chosen on validation only	same weights exist. Dropping the ensemble costs about -0.06 mAP@0.5 on hyperpigmentation test and about -0.02 on acne	3x faster, 3x less memory, simplest	picking the "best" of three is itself noisy (seeds differ by up to 0.035 on one set); not checked whether the validation pick was best on the fresh test
2. Current 3-seed fusion	table above	best measured accuracy, less seed luck, no retraining	3x latency and memory; mini-PC speed NOT MEASURED
3. One multi-class detector	the earlier combined 3-class detector scored acne 0.22 vs 0.41 for the specialist and was removed	one file, one pass	evidence so far says it loses accuracy; needs new unified labels and training; do not assume it matches specialists
4. Distillation / compression	no experiment	possible one small model imitating the ensemble	non-trivial for detection; needs training and a new evaluation; future work
5. Merge three seeds into one ONNX (tested)	`shared/onnx_merge.py` , numerically exact (difference 6e-5)	one file, same output layout	**scores lower than box fusion** (averaging raw outputs misaligns anchors), so not installed

The MobileNetV2 classifier is independent of this decision. An eczema detector with three seeds would add three more files, so the same trade-off applies.

6.4 Hardware

The detectors run on **CPU** with ONNX Runtime (`LesionDetector.cs` , `GraphOptimizationLevel.ORT_ENABLE_ALL` , configurable intra-op threads). The GT 1030 2 GB is not a realistic target; not tested.

- **Measured (development laptop, 2 CPU threads):** about 0.85 s per model per 800 px photo; acne ensemble about 2.5 s; all six models about 3.1 s per photo. Earlier single-model figures: 0.29 s per frame (acne, laptop, 2 threads) and 131 ms in the `.json` for the acne model on the export machine.

- **NOT MEASURED:** anything on the mini PC (`shared/benchmark_pc.py` exists and has not been run there), memory use, model load time, a full three-view scan. Arithmetic only: if all six models ran on all three photos, about 9 s on the laptop settings; the real figure depends on how the app schedules detectors per view. Storage 6 x 44.8 MB, about 270 MB.
- Running inference sequentially after capture, not during the live preview, is a software scheduling choice and does not change the approved objectives; whether the app already does this was not traced.
- **Fair benchmark plan (no retraining):** same photos on the mini PC, 3 warm-up runs then 30 timed runs; report median and 95th percentile of (a) model load time, (b) latency for 1, 2 and 3 seeds, (c) peak RAM, (d) full three-view scan time with classifier, Grad-CAM++ and detectors, (e) CPU temperature and throttling over 20 consecutive scans. State the acceptable scan time before looking at the numbers.

6.5 Recommendation

Keep the three-seed ensembles for the thesis, and run two cheap checks that need no retraining before the defense: (a) the mini-PC benchmark, (b) a paired bootstrap of ensemble vs each single seed on existing predictions. Decision rule: keep the ensemble for a condition if the paired difference vs the validation-chosen single seed is positive with the 95 % range above 0, or the extra latency is acceptable; otherwise offer a single-model mode for that condition. Acne is the first candidate to simplify.

Rank by	1st	2nd	3rd	4th
Scientific defensibility	3-seed ensemble	best single seed	multi-class (evidence against)	distillation (untested)
Deployment efficiency	multi-class (if it worked)	best single seed	distillation	3-seed ensemble
Reproducibility	best single seed	3-seed ensemble	multi-class	distillation
Implementation risk (lowest first)	best single seed	3-seed ensemble (built)	multi-class	distillation
Expected performance	3-seed ensemble	best single seed	multi-class	distillation (unknown)

Missing evidence: paired significance; mini-PC latency, RAM and load time; tone-stratified results; false alarms per seed; real-scan evaluation in the app.

Part 7. Code reference

7.1 Repository layout

```
SkinDevApp/             the WPF app (C#). Model\ = classifier + 3 acne + 3 hyperpigmentation ONNX
files
gradcam_service.py      Grad-CAM++ service (must stay in the root: the app looks for it here)
precisionskin_best.keras, labels.json, requirements_gradcam.txt, start_gradcam_service.bat
acne/                   notebooks/ scripts/ docs/ data_lists/ evidence/
hyperpigmentation/      notebooks/ scripts/ docs/ data_lists/
eczema/                 notebooks, review tool, team guide (a teammate trains the eczema detector)
skintone/               Monk Skin Tone model: notebook, mst_skintone.py, download_scin.py, paper
section
gradcam/                Grad-CAM studies and the tool that upgrades existing scans
shared/                 tools used by several conditions, notebooks/, docs/
docs/                   this document
```

Big image data stay in `Downloads\PrecisionSkin\...` (participant photos are never committed). Training happens in WSL (`~/PrecisionSkinV1` , environment `.venv_det`). Python tools find each other through a path line at the top of each script. Model files (`*.onnx` , `*.pt`) are on disk; the researcher's own commits also track the ONNX models.

7.2 The app (C#, `SkinDevApp/`)

AI and detection

File	What it does
`Aiengine.cs`	application-wide singleton around the ONNX classifier (same static-class pattern as `DatabaseHelper`)
`Onnxinference.cs`	fast CPU classifier backed by `precisionskin.onnx` ; preprocessing byte-identical to the Grad-CAM service
`Predictionresult.cs`	result of one classification; class order fixed (Acne, Hyperpigmentation, Eczema, Normal)
`Predictionstabilizer.cs`	smooths live predictions over frames
`LesionDetector.cs`	YOLOv8 ONNX detector. Loads specialists `lesion_detector_<class>.onnx/.json` ; extra seeds `_m2` , `_m3` as `Peers` ; `EnsembleSize = 1 + Peers.Count` ; when peers exist runs all and fuses boxes (weighted box fusion IoU 0.5); NMS IoU and thresholds from the `.json` ; a broken specialist falls back automatically; boxes outside the detected face are dropped
`ModelInfo.cs` , `ModelInfoPage.xaml.cs`	traceable model identity (file, SHA-256, ensemble, thresholds) shown to researchers and stored with each capture

Explainability

File	What it does
`Gradcamservice.cs`	client of the local Grad-CAM++ service (`ExplainAllClassesAsync`, v2.0)
`GradCamEnvironment.cs`	makes the service portable: inspects the PC (Windows version, CPU, RAM, GPU, free port 8765), finds Python, starts or connects to the service
`ClassHeatmapSet.cs`, `ClassHeatmapRenderer.cs`, `Heatmaprenderer.cs`, `ClassPalette.cs`	per-class heatmaps, class colours (Eczema yellow/orange, Normal green, etc.), client-side blending
`RegionAttribution.cs`	approximate facial-region description of where a class map is strong (not a diagnosis)
`Livegradcamcontroller.cs`	decoupled loops (classifier, stabiliser, Grad-CAM) plus auto-capture

Scanning

File	What it does
`ScanStateMachine.cs`	pure, thread-safe scan flow: Live, Scanning, Stabilizing, HoldStill, StableCapturing
`ScanView.cs`, `MultiViewSession.cs`	the three standardised views and one multi-view session
`ViewPoseEstimator.cs`	checks the patient is really showing the requested view (YuNet landmarks)
`FrameQuality.cs`	guidance state and face-box layout
`ScanPipeline.cs`	final analysis of one captured frame (auto, manual snap or file upload)
`CameraDiscovery.cs`	finds the physical webcam (selected camera, then Logitech BRIO, then others; virtual and infrared cameras last)
`CaptureArchive.cs`, `CaptureRecord.cs`, `GalleryModel.cs`	one folder per capture with `record.json` and honest metadata; loading a saved scan
`Imageinterop.cs`	conversions between WPF BitmapSource, System.Drawing and OpenCvSharp Mat

Study and screens `DatabaseHelper.cs` (SQLite), `RegistrationPage`, `ConsentPage` (scanning blocked until consent), `PreparationPage`, `ClientDashboardForm` (guided auto-capture), `ResearcherResultWindow`, `ResearcherVerificationWindow` (AI prediction beside two independent human layers; append-only), `PatientRecordPage` (search by ID or name, Monk Skin Tone field), `ComparisonGalleryWindow`, `PrintReportWindow`, `ResearchDashboardPage`, `SettingsPage` (camera, accounts, change history; image-analysis, heatmap and lesion values are fixed for the study in `ScanSettings`: thresholds acne 0.275, hyperpigmentation 0.225), `LoginForm`, `MainWindow` (full-screen kiosk). `PrecisionSkinReferences.cs` is a reference implementation kept for comparison.

7.3 Shared Python tools (`shared/`)

Script	What it does
`lesion_box_tool.py`	local web box editor used for all boxing (standard library only, nothing uploaded; X = no lesion; S saves)
`box_progress.py`	shows how far each annotator is
`lesion_exp.py`	controlled experiment runner: each experiment is a spec (id, condition, data recipe, model, imgsz, seed, overrides); datasets read through file lists; originals never touched; runs in `lesion_experiments/runs/<id>/` ; validation photos only; registry `EXPERIMENTS.csv` with data fingerprints
`lesion_diagnose_data.py` , `lesion_diagnose_models.py`	read-only diagnosis of datasets (box size, density, overlap, border, duplicates) and of models (training curves, PR and F1 curves, recall by box size, false and missed boxes)
`lesion_alt_metrics.py` , `lesion_exp_usage_metrics.py`	metrics matching how the app is used: photo level (AUC, sensitivity, specificity), count level, 3x3 zone level
`negatives_eval.py`	how often the detectors box clear skin
`install_specialist_detector.py`	exports `best.pt` to ONNX (opset 12, fixed size), writes the `.onnx` and `.json` into `SkinDevApp/Model/` (classes, imgsz, nms_iou, SHA-256, source weights, metrics)
`onnx_merge.py`	merges N YOLOv8 ONNX files into one (renamed graphs, shared input, mean or score-weighted mean); verified exact but lower quality, not installed
`build_report_notebooks.py`	writes the training-and-results notebooks per condition (holds the `TOOLBOX` helper code: rd, iou, raw_preds, fuse, score, best_conf, prf, zone_f1, f1_by_iou, bootstrap_f1, curves)
`build_evidence_notebook.py` , `run_notebook_evidence.sh`	read-only evidence notebook (environment, who did what, dataset counts and labelling logs, args.yaml, history, registry, installed SHA-256, evaluation reproduction, missing items, verification commands)
`benchmark_pc.py`	speed on the mini PC (CPU, RAM, classifier, detector, one full capture); not run on it yet
`md_to_html.py`	Markdown to a printable A4 HTML page (used to produce this PDF)
`backup_training_to_windows.sh`	copies training outputs to Windows

7.4 Acne scripts (`acne/scripts/`)

Dataset: `merge_acne_dataset.py` (v1 to v4), `make_more_faces.py` , `draft_more_faces.py` , `make_independent_set.py` , `make_acne_fresh_test.py` , `make_acne_more_boxing.py` , `make_acne_reliability.py` , `acne_second_look.py` (a copy of training photos with the model's suggested extra boxes). Helper: `acne_predraft.py` (train on own boxes, then draft). Audits: `acne_annotation_audit.py` , `acne_reliability.py` , `acne_box_scale_check.py` , `acne_box_margin_check.py` , `acne_verdict_checks.py` , `acne_gate_rule.py` , `acne_fresh_negatives_by_source.py` . Evaluation: `acne_final_report.py` , `acne_compare_report.py` , `acne_ensemble_val.py` , `acne_ensemble_test.py` , `acne_candidate_full_table.py` . Training and notebooks: `train_acne_v5.py` , `build_acne_comparison_notebook.py` , `build_phase_notebooks.py` (writes only the

notebooks named on the command line), launchers `run_acne_audit.sh`, `run_acne_c.sh`, `run_notebook_phase2.sh`, `run_notebook_acne_comparison.sh`.

7.5 Hyperpigmentation scripts (`hyperpigmentation/scripts/`)

`hyperpig_autolabel.py` (one-class dataset, indexing, pseudo-labelling, scoring), `hyperpig_predraft.py` (helper trained on own boxes), `hyp_v2_transform.py` (old boxes to rule v2 by code, parameters fit on pass A and checked on pass B), `make_hyp_reliability_v2.py` and `hyp_reliability_v2.py` (blind passes of 30 photos and agreement), `make_hyp_v2_relabel.py` and `make_more_boxing.py` (review folders), `hyp_v2_val60.py` (60-photo validation and threshold), `hyp_v2_train_consistency.py` (drift and correction checks), `hyp_ensemble_tta.py`, `hyp_final_eval.py` (frozen test evaluation), `hyp_full_results.py` (complete results file; the stored one still describes the H20 stage and could be regenerated for H25), `run_hyp_h25seeds.sh`.

7.6 Notebooks

- Classifier: `Pipeline V5 - With Facial Eczema.ipynb`.
- Detectors (final): `acne/notebooks/PrecisionSkin_Acne_Training_and_Results.ipynb`, `hyperpigmentation/notebooks/PrecisionSkin_Hyperpigmentation_Training_and_Results.ipynb`, acne `Phase1_Experiments`, `Phase2_FinalTraining`, `Phase3_FreshTest`, `Acne_Data_Comparison`.
- Evidence: `shared/notebooks/PrecisionSkin_Training_Evidence_Acne_and_Hyperpigmentation(.ipynb / _WITH_OUTPUTS.ipynb)`, and the researcher's executed `acne/evidence/Pipeline V5 - Lesion Detector__EXECUTED_by_researcher_2026-10-10.ipynb`.
- Earlier: Localization Dataset Audit, Detector Export and Train, Eczema Draft and Review, Eczema v2 Pipeline and Finish, MST training notebook.

7.7 How a detector is trained, evaluated and installed (step by step)

1. Box photos with `lesion_box_tool.py` (copies only; originals untouched). 2. Merge into a YOLO dataset with leak checks. 3. Train through the notebook (3 seeds, `last.pt` resume). 4. Choose the checkpoint and the confidence threshold on validation. 5. Fuse the 3 seeds (weighted box fusion). 6. Score the frozen test once, write the lock file. 7. Measure false alarms on clear skin. 8. Install with `install_specialist_detector.py` (ONNX opset 12, fixed 800 px) for each seed. 9. Rebuild the app and confirm the app's C# detector reproduces the offline score. 10. Record the run in `EXPERIMENTS.csv` and the evidence notebook.

7.8 Reproducibility records

Audit reports in `localization_dataset/01_audit_reports/`; datasets `detector_work/dataset_v1`, `dataset_v2` with `dataset_manifest.csv` (split, source, group, box origin per image); runs `detector_work/runs/`; exports with SHA-256; `lesion_experiments/EXPERIMENTS.csv` for every detector run; run records `args.yaml` and `results.csv`.

Part 8. Limitations (state these plainly)

1. **Grad-CAM++ is attribution, not localisation.** The classifier makes a global image-level decision; highlighted regions need not be diseased skin.
2. **Class maps are similar** (correlation 0.95 to 0.99) because the classifier uses largely shared facial features. This is a model property, not a bug.
3. **Detector accuracy is modest.** Acne fresh-test mAP@0.5 0.336 and hyperpigmentation 0.508; strict localisation mAP@0.5:0.95 is 0.09 and 0.21. Boxes are candidate locations, not a diagnosis or a count; lesions are missed (recall about 0.47 to 0.56).
4. **Label quality.** One annotator; same-annotator agreement F1 0.30 for acne and about 0.5 for hyperpigmentation; box-level scores above that level cannot be demonstrated. Public sources were inconsistent (very large boxes for pigment, tiny boxes for acne).
5. **False alarms.** Acne boxes 39 % of no-acne photos (15 % on clear skin); hyperpigmentation about 32 % on clear skin. The acne detector is a spot marker, not acne-specific.
6. **The classifier confuses acne with eczema** (about 13.5 % of acne photos).
7. **Eczema is a pilot class:** the earlier extension did not help; 32 test photos; the teammate's detector is not installed.
8. **Facial status is partly assumed:** six public sources were declared facial from small visual samples.
9. **Domain gap.** Training mixes close-ups, crops and full faces from public data; live kiosk frames are mostly full faces with different lighting and resolution. Only informal checks on live captures; no real-scan evaluation of the detectors yet.
10. **Skin tone.** No per-tone evaluation for the classifier or the detectors; datasets have no Fitzpatrick labels.
11. **Test-set history.** The acne development test guided earlier decisions and has been scored several times (hence the fresh test); the independent set has 41 photos and the fresh set 100 photos with 203 boxes, so ranges are wide.
12. **Scores under rule v2 are not comparable** with scores under the first hyperpigmentation rule.
13. **Duplicate handling is heuristic** (thumbnail correlation plus names); calibrated and visually checked, not perfect.
14. **Speed.** Mini-PC latency, memory and load time are NOT MEASURED.
15. **Research prototype; not a medical device.** No claim of clinical accuracy.
16. **AI assistance.** An AI coding assistant helped write code and ran many experiments on the researcher's computer at their request; this is documented in the evidence notebook.

Part 9. Ethics and data handling

- **Ensemble learning is ethically neutral.** Ethics depends on use, not on the method. What matters is honesty about accuracy, limits and consent. An acceptable technique is not a clinically validated application.
- **Intended use.** Research prototype and AI-assisted screening aid. Do not claim diagnosis, dermatologist-level performance, regulatory approval or clinical accuracy.
- **Risks to state.** False alarms (boxes are candidates, not findings); misses (a clear result must never be read as "healthy"); the fused confidence is a mean of three confidences, not a calibrated probability (do not display it as "% chance of acne"); skin-tone fairness is unmeasured; three models triple inference energy per scan (negligible at kiosk scale, but a cost with no measured clinical benefit).
- **Transparency and reproducibility.** Fixed seeds, dataset fingerprints, SHA-256 per ONNX, locked test scoring, evidence notebook, experiment registry. Disclose AI assistance in the way the school's policy requires.
- **Data.** Training data are public CC BY 4.0 Roboflow datasets: keep attributions (project, author, URL) in the references. Webcam captures are real participants' faces: keep them out of git and online tools, use local annotation tools, follow the consent terms. Reports use participant IDs (PS-xxxx), not names. Do not copy participants' photos to other computers without the consent terms. Scanning is blocked until consent is recorded.
- **Documentation to present:** intended-use and non-use statement, consent and data handling, the limitation list above, validation evidence from Parts 2 to 6, and the dermatologist-review workflow built into the app.

Part 10. Defense preparation

10.1 The 60-second answers

- **What is it?** A kiosk that screens a face photo: a MobileNetV2 classifier (Acne, Hyperpigmentation, Eczema, Normal), Grad-CAM++ to show what influenced the classifier, and YOLOv8 detectors that propose candidate lesion boxes. A screening aid, not a diagnosis.
- **How were the detectors trained?** One-class YOLOv8s, fine-tuned from COCO weights on photos the researcher boxed by hand with one written rule, 800 px, up to 100 epochs, three seeds. Choices and thresholds on validation only; test scored once; final detector averages three seeds.
- **What is the final architecture?** MobileNetV2 four-class classifier (+ Grad-CAM++ as attribution) and, per condition, a 3-seed YOLOv8s ensemble (800 px, weighted box fusion IoU 0.5, thresholds 0.275 acne and 0.225 hyperpigmentation), boxes filtered to the face; eczema detector pending.

10.2 Classifier questions

- **Why MobileNetV2?** Small (5 MB ONNX), fast on a CPU-only kiosk, strong pretrained features, enough accuracy.
- **Why 4 classes and no mimickers?** The approved objectives cover acne, eczema, hyperpigmentation and a normal class.
- **Is it memorising duplicates?** The split is by duplicate group; a leak-free re-evaluation still gives accuracy 0.909 and macro-F1 0.912.
- **Why facial photos only?** The kiosk takes face photos; the `facial_only` run is the deployed one.
- **Why 5 seeds and the test once?** To show the result is not one lucky run, and to avoid choosing the model on the test set; the seed was chosen on validation.
- **Why is eczema precision low?** Acne photos get mislabelled as eczema (shared redness features); class weights raise eczema recall at the cost of precision. The 0.65 confidence gate filters about 25 % of these errors.
- **What does Grad-CAM++ show?** What influenced the classifier's score. Attribution, not localisation.
- **Skin tones?** Not evaluated per tone; Fitzpatrick model discontinued; MST model planned, not trained.
- **Live camera?** Only informal checks; a participant pilot with dermatologist verification is the real test.

10.3 Detector questions

- **Why YOLOv8, and why the small model?** One-stage, fast on CPU after ONNX export, simple to train, well documented. A medium model gave no better score (acne 0.426 vs 0.425; hyperpigmentation 0.305 vs about 0.30). Do not claim it is the best possible architecture; other detector families were not compared.
- **What does mAP@0.5 mean?** The share of true lesions found with a box overlapping the true box by at least 50 %, averaged over confidence levels. mAP@0.5:0.95 needs tighter overlap and is always lower, especially for tiny objects.
- **Why is acne mAP only about 0.34 to 0.44?** Tiny spots, dense photos, and label noise (the researcher agrees with their own earlier boxes at F1 0.30). The detector is a candidate marker, not a counter.
- **Why 0.508 for hyperpigmentation but 0.336 for acne?** Different tasks and rules. Hyperpigmentation uses large whole-patch boxes under a reproducible rule, which are easier to match than tiny spots.

- **Why are localisation scores lower than classification scores?** Localisation needs exact boxes on soft, tiny or diffuse lesions; classification is one label per photo, much easier to agree on. Zone-level results are higher (acne 0.70 to 0.83, hyperpigmentation 0.61).
- **Is the test set independent?** Hyperpigmentation: the 100 test photos were never used for training or choosing and were scored once. Acne: the development test influenced early decisions (say so); a 41-photo independent set and a 100-photo fresh set were each scored once.
- **Do the detectors prove disease?** No; the classifier and the dermatologist's verification are separate.
- **What if classifier and detector disagree?** The screening result is the classifier's (above the 0.65 gate); boxes are supplementary.
- **Where did the data come from?** The researcher's own boxing (tool logs: about 12 hours for acne and 6 hours for hyperpigmentation) plus public Roboflow Universe sources (CC BY 4.0) for extra training photos.
- **Who did the work?** The researcher boxed the data and ran the final acne training; an AI coding assistant helped write code and ran many experiments at their request. Say this plainly.
- **How fast?** About 0.5 to 1 s per model on a laptop CPU, about 3 s for all six models. Mini-PC speed is not measured.

10.4 The three-model questions

1. **Why three models for one condition?** Simple: "Training has randomness, so each copy makes slightly different mistakes. Averaging three copies cancels some of them, like asking three people instead of one." Technical: seed ensemble reduces variance; weighted box fusion at IoU 0.5; measured gain +0.058 mAP@0.5 on hyperpigmentation test and +0.016 on the acne fresh test.
2. **Doesn't it mean the final model is unreliable?** No. The ensemble is the final model, frozen with its threshold before testing; the seed spread (std about 0.02) is the reliability figure we report.
3. **Why not just choose the best model?** We can, and it is 3x faster. The "best" is picked on a small validation set and can be wrong; the ensemble was never worse than any member in five comparisons. For acne the gain is small and we say so.
4. **Is this overfitting?** No. Different seeds on the same data are not memorisation. Overfitting risk comes from tuning on the test set; we chose thresholds on validation, scored tests once, and rejected the acne 960 px change when its gain failed on fresh data.
5. **Is ensemble learning scientifically recognised?** Yes (deep ensembles; weighted box fusion for detection).
6. **Is it ethical to use multiple models in a medical-related app?** The method is acceptable; the claim must stay modest: research screening aid, consent, local data, dermatologist review, disclosed false-alarm and miss rates.
7. **How can you justify the added computation?** Hyperpigmentation: +0.058 for 3x compute, supported. Acne: +0.016 to +0.02, weaker. Mini-PC latency is still to be measured, and we state that openly.

10.5 Things not to say

- "The system is clinically accurate / diagnoses disease." It is a research prototype.
- "Grad-CAM++ shows where the disease is." It shows what influenced the classifier.
- "The model works on all skin tones." It was not evaluated per tone.
- "0.508 is comparable to published detectors." It is under the researcher's own box rule on their own test photos.

- "I trained everything myself." State what was done by whom.
- "The ensemble is statistically proven better." No paired test has been run.

10.6 Limits to state first

One annotator; modest detector scores for tiny acne spots; box-rule dependence for hyperpigmentation; the classifier confuses acne with eczema; false alarms of about 30 to 40 % on clear photos; no per-skin-tone evaluation; domain gap between public photos and kiosk frames; mini-PC speed not yet measured; eczema detector in progress by a teammate.

10.7 Where to point for proof

Classifier: the executed classifier notebook and `runs/facial_only/`. Acne final training: `acne/evidence/...EXECUTED_by_researcher_2026-10-10.ipynb`. Detector records: the evidence notebook, `lesion_experiments/EXPERIMENTS.csv`, the `.json` files next to each ONNX.

Part 11. Mistakes, corrections and open items

11.1 Mistakes and corrections (so the record is honest)

- Early on the assistant said drafting was not faster at 119 photos; measured later: 14.2 s per photo (faster).
- Once killed too many python processes and stopped the researcher's box tool (labels were safe; backed up).
- The "doubled or nested box" rule was wrong for close-ups and touching neighbours; fixed (only 5 of 46 flagged photos were truly doubled).
- The plan to "draft 400 new facial photos" was wrong: the facial pool was already used up; the drafted non-facial photos hurt the score.
- A hyperpigmentation size comparison was judged valid until the diagnosis showed the n and m runs had stopped after 7 and 9 epochs.
- The acne 0.448 result was a lucky seed; the honest figure is 0.413.
- WSL crashed several times; restarted with the researcher's agreement; jobs are resumable.
- An acne-negatives training was started once before the researcher wanted it; stopped, then re-run properly (A1).
- A log file (rep3.log) was deleted without asking; an evaluation was once described as started before it had; the learning-rate gain was overstated until all seeds were in; the assistant once told the researcher to close "extra kernels" that were really dataloader workers of the single training kernel.
- Phase 3 notebook bug: the first fresh-test run used the wrong folder (36 positives, no negatives, false-alarm rate undefined); fixed, first lock archived, corrected result reported.
- The phase-notebook builder once rebuilt all three notebooks and reset the Phase 2 configuration; fixed so it builds only the notebooks named.
- The acne `.json` still carries the older single-seed `test_mean`; the correct figure is in its `note`.

11.2 Open items

1. A real scan in the app to confirm both detectors draw boxes.
2. Mini-PC measurement with `benchmark_pc.py` (classifier and both detectors, one seed vs three, full three-view scan).
3. Paired bootstrap of ensemble vs single seeds on existing predictions.
4. A one-day rule-v2 re-check (pass C) and an independent set under rule v2 for hyperpigmentation; clear-skin specificity for hyperpigmentation.
5. Regenerate `HYPERPIGMENTATION_FULL_RESULTS.md` for H25.
6. Skin tone: download MST-E and SCIN, verify licences, run the audit and the notebook (Appendix B).
7. Eczema detector (teammate), with a hand-boxed eczema test set.
8. UI polish (needs screenshots); study forms still need a wording check (mimicker, hardware description, Fitzpatrick claims).
9. Add exact names, URLs and citations of all public sources to the references.
10. Compile final thesis documentation once the whole system is finished.

Appendix A. Run the whole app on a new laptop

This sets up everything: the WPF app, the Grad-CAM++ service and the lesion detectors. About 1 hour the first time. Windows 10 or 11 (64-bit) with a webcam and internet. The participant data, scans and database of the original machine are **not** on GitHub; the new copy starts empty.

A.1 Install the tools (once)

Tool	Notes
Git	https://git-scm.com/download/win (default options)
Visual Studio 2022 or newer, Community	workload **".NET desktop development"**, plus the individual component **".NET Framework 4.7.2 targeting pack"**
Python 3.12 (3.11 also works; **not 3.13 or 3.14**)	tick **"Add python.exe to PATH"** and keep the **py launcher**

Check in PowerShell: `git --version` and `py -3.12 --version`.

A.2 Get the code

```
cd C:\Users\<you>\source\repos
git clone https://github.com/AndyVal0611/SkinDevApp
cd SkinDevApp
git checkout master
```

(If already cloned: `git checkout master` then `git pull`.) The folder contains both kinds of ONNX models (`SkinDevApp\Model\`) and the Keras model (`precisionskin_best.keras`).

A.3 Set up the Grad-CAM++ service (once, about 10 minutes)

Inside the SkinDevApp folder (the one with `start_gradcam_service.bat`):

```
py -3.12 -m venv .venv
.venv\Scripts\python -m pip install --upgrade pip
.venv\Scripts\python -m pip install -r requirements_gradcam.txt
```

This downloads TensorFlow (large). Then **double-click** `start_gradcam_service.bat`. A black window opens; loading takes up to a minute; when it prints **READY** the service runs on `http://127.0.0.1:8765`. Keep it open while using the app. Quick check: `http://127.0.0.1:8765/health` should show `"ok": true`.

A.4 Build and run the app

1. Open `SkinDevApp.slnx` in Visual Studio. 2. Choose **Debug** and **x64** (not "Any CPU", not x86). 3. **Build > Build Solution** (`Ctrl+Shift+B`); the first build downloads NuGet packages. 4. Press **F5**.

First sign-in: there is no default account; the login screen asks you to create the first researcher account (at least 8 characters). It only exists on this laptop.

A.5 Try a scan

Register a test participant (a teammate, not a real patient) and record consent. On the preparation screen check the classifier is ready (the Grad-CAM++ service is optional but needed for heatmaps). Scan Front, Left, Right with good, even light, facing the camera and holding still. On the Results page use the toggle: **Original / Grad-CAM++ / Localization / Combined**. Researcher screens: Comparison Gallery, Generate Research Report, Settings.

A.6 If something goes wrong

Problem	What to do
Build error about missing packages	Check internet, allow NuGet restore (Tools > NuGet Package Manager), or right-click the solution > Restore NuGet Packages
Build says the file is locked	Close any running copy of the app, then rebuild
Platform box says "Any CPU"	Choose **x64**
`python` is not recognised	Use `py -3.12`
Service window says "No Python with TensorFlow was found"	Run the three commands in A.3 inside the SkinDevApp folder
Port 8765 already in use	Another copy of the service is running; close it and start again
"Grad-CAM++ unavailable" on Results	The service window is not open or still loading; scan, scores and localization still work
No camera found	Windows Settings > Privacy & security > Camera: allow apps and desktop apps; close other apps using it; pick the camera in Settings
"Localization not available" on an old scan	Normal: scans made before the feature have no localization

A.7 Good to know

Localization boxes are candidate lesion locations from a research detector: not a diagnosis, not a lesion count; lesions can be missed; eczema is a pilot class. Do not copy participants' photos to other computers or online tools without the study's consent terms. Training and notebooks are not needed to run the app.

A.8 Sharing one database between laptops

The database (`lumyvue_db.sqlite`) and the scan images (`Captures`) are never pushed to GitHub. To work on the same participants from two laptops, put them in a folder that both laptops sync (Google Drive, OneDrive, Dropbox):

1. Make a folder such as `LUMYVUE_Data` , share it, and wait until it has fully synced.
2. On **each** laptop create `%LOCALAPPDATA%\LUMYVUE\data_folder.txt` whose first line is the full path of the synced folder (or set the environment variable `LUMYVUE_DATA_DIR`).
3. Start the app. The first laptop to start copies its existing database into the folder; after that both read and write the same database and `Captures` folder.

Rules: only one person has the app open at a time; close the app and wait for the sync to finish before the other person opens it (two at once, or before the sync finishes, can corrupt the database); keep a backup copy of `lumyvue_db.sqlite`.

Appendix B. Monk Skin Tone (MST) model plan (separate research model; not trained yet)

Datasets. *MST-E* (Monk Skin Tone Examples, Google Research): reference images of 19 people across the 10 MST shades (1,515 images and 31 videos per the NeurIPS 2023 paper), released to help practitioners and annotators understand the scale; the shade of each person was approved by the scale's creator. Many photos of few people, so it is a reference or supplementary set, not the sole training set. *SCIN* (Skin Condition Image Network, Google Research and Stanford Medicine): crowd-contributed photos of skin, nail and hair conditions with layperson-**estimated** MST labels (and dermatologist-estimated Fitzpatrick labels). Many photos show body parts, so the set is filtered (head and neck cases only, one photo per case). Verify the exact column names and the licence of both datasets against the downloaded files; the SCIN repository lists a few duplicate images and a few cases without a condition label.

Independence. The MST model is separate from the condition classifier and the detectors; its output never changes a condition result or a box. Skin-tone labels used to evaluate the classifier per tone group must come from rater or dermatologist assessments (the app's Monk Skin Tone field in registration), not from this model, until it is validated.

Method safeguards (lessons from the Fitzpatrick failure). Group split (all photos of one person or SCIN case in one split); labels only from raters; no hue or saturation augmentation; results reported as exact shade, within one shade (rater disagreement is about one step), and three bands (1 to 3, 4 to 6, 7 to 10), with a 95 % range by resampling people, a brightness stress test (+/-25 %), and per-source results. Code: `mst_skintone.py`, notebook `PrecisionSkin_SkinTone_MST_Training.ipynb` (smoke-tested on fake data only). App side: optional Monk Skin Tone (1 to 10) and who rated it in registration, shown on the patient record, changes audited.

Known limits. SCIN tone labels are layperson estimates; MST-E has few people; both are phone or studio images, not ring-light kiosk captures, so a small set of kiosk captures rated by two raters against the MST card is needed before any claim about the deployed system; shades at the dark end are rare in SCIN.

Appendix C. Published context for acne detection and paper wording

Checked against the PDF of Huynh et al. 2022, Diagnostics 12(8):1879, doi 10.3390/diagnostics12081879.

Study	What was detected	Data	Model	mAP as reported
Huynh et al. 2022 (AcneDet)	4 types: blackheads/whiteheads, papules/pustules, nodules/cysts, acne scars	1,572 smartphone images, 41,859 lesions, labelled by 4 dermatologists, 70:30 split	Faster R-CNN ResNet50	0.54 overall; per type 0.40 / 0.64 / 0.68 / 0.44
Min et al. (ACNet), as cited in that paper	general acne, no types	18,983 acne objects (ACNE04)	ACNet	0.205
Kuladech et al., as cited in that paper	4 types	15,917	Faster R-CNN / R-FCN	0.233 / 0.283

Study	What was detected	Data	Model	mAP as reported
This study	general acne, one class	1,145 train photos, 11,621 boxes, one annotator	YOLOv8s, 800 px	0.34 (fresh test) to 0.44 (development test), mAP@0.5

Reading it: the paper's lowest APs are the small, low-contrast types and the highest the large, red ones; a single "acne" class mixes all types and is dominated by small ones. Weighting the paper's non-scar types by lesion counts gives about 0.44 (a back-of-envelope calculation, not a figure from the paper). The paper does not state its mAP IoU threshold or whether its split keeps one person's selfies together, and datasets, annotators and image sizes differ, so this is context, not a ranking.

Wording you can use (updated to the final installed models).

> Separate condition-specific YOLOv8s detectors were trained for acne and hyperpigmentation on researcher-boxed photos and evaluated on independently hand-boxed test photos that were never used for training or threshold selection. The hyperpigmentation detector (three-seed weighted box fusion, trained under a written annotation rule) reached mAP@0.5 = 0.51 and F1 = 0.52 on 100 held-out photos, up from 0.18 for the detector it replaced under the earlier rule. The acne detector (three-seed fusion) reached mAP@0.5 = 0.44 on a development test set and 0.34 on a fresh untouched set of 100 photos, and puts boxes on 39 % of photos without acne. Box-level scores are capped by the annotator's agreement with themselves (F1 0.30 for acne and about 0.5 for hyperpigmentation at IoU 0.5), so face-zone results are also reported. The detectors propose candidate lesion boxes; they are not a diagnosis, and Grad-CAM++ is reported separately as model attribution, not localisation.